

正则表达式

什么是正则表达式

正则表达式是一种用来描述字符串匹配模式的语法规则，就像一把“文本过滤器”或“高级搜索工具”。

能解决哪些问题？（结合实际场景）

场景	正则能做什么
日志分析	从大量日志中提取 IP、时间戳、错误信息等关键字段
输入校验	判断手机号、邮箱、身份证号是否符合格式
数据提取	从 HTML 页面中提取链接地址、图片 URL、标题等
文本替换	批量替换代码中的函数名、变量名，或者清洗脏数据
爬虫开发	抓取网页时，用正则快速提取内容
批量重命名	使用正则+脚本改名一堆图片、视频、文件名等
配置审计	查找配置文件中被注释掉的行、未配置项等

正则表达式就像在茫茫人海中寻找你要的人，它提供一组“筛选规则”：只要长得像这个样子、有这个特征的人就挑出来。
它不只“看起来像”，而是按规则“精准匹配”。

正则表达式简史

正则表达式的起源，可以追溯到，早期神经系统如何工作的研究。在 20 世纪 40 年代，有两位神经生理学家（Warren McCulloch 和 Walter Pitts），研究出了一种用数学方式来描述神经网络的方法。

1956 年，一位数学家（Stephen Kleene）发表了一篇标题为《神经网络事件表示法和有穷自动机》的论文。这篇论文描述了一种叫做“正则集合（Regular Sets）”的符号。

随后，大名鼎鼎的 Unix 之父 **Ken Thompson** 于 1968 年发表了文章《正则表达式搜索算法》，并且将正则引入了自己开发的编辑器 qed，以及之后的编辑器 ed 中，然后又移植到了大名鼎鼎的文本搜索工具 grep 中。自此，正则表达式被广泛应用到 Unix 系统或类 Unix 系统 (如 macOS、Linux) 的各种工具中。

随后，由于正则功能强大，非常实用，越来越多的语言和工具都开始支持正则。不过遗憾的是，由于没有尽早确立标准，导致各种语言和工具中的正则虽然功能大致类似，但仍然有不少细微差别。

于是，诞生于 1986 年的 POSIX 开始进行标准化的尝试。POSIX 作为一系列规范，定义了 Unix 操作系统应当支持的功能，其中也包括正则表达式的规范。因此，Unix 系统或类 Unix 系统上的大部分工具，如 grep、sed、awk 等，均遵循该标准。我们把这些遵循 POSIX 正则表达式规范的正则表达式，称为 **POSIX 流派**的正则表达式。

在 1987 年 12 月，Larry Wall 发布了 Perl 语言第一版，因其功能强大一票走红，所引入的正则表达式功能大放异彩。之后 Perl 语言中的正则表达式不断改进，影响越来越大。于是在此基础上，1997 年又诞生了 PCRE——**Perl 兼容正则表达式**（Perl Compatible Regular Expressions）。

PCRE 是一个兼容 Perl 语言正则表达式的解析引擎，是由 Philip Hazel 开发的，为很多现代语言和工具所普遍使用。除了 Unix 上的工具遵循 POSIX 标准，PCRE 现已成为其他大部分语言和工具隐然遵循的标准。

之后，正则表达式在各种计算机语言或各种应用领域得到了更为广泛的应用和发展。**POSIX 流派**与**PCRE 流派**是目前正则表达式流派中的两大最主要的流派。

正则表达式流派

目前正则表达式主要有两大流派（Flavor）：**POSIX 流派**与**PCRE 流派**。

POSIX 流派

先简要介绍一下 POSIX 流派。POSIX 规范定义了正则表达式的两种标准：

- **BRE 标准**（Basic Regular Expression 基本正则表达式）；
- **ERE 标准**（Extended Regular Expression 扩展正则表达式）。

BRE 标准和 ERE 标准

早期 BRE 与 ERE 标准的区别主要在于，BRE 标准不支持量词问号和加号，也不支持多选分支结构管道符。BRE 标准在使用花括号，圆括号时要转义才能表示特殊含义。BRE 标准用起来这么不爽，于是有了 ERE 标准，在使用花括号，圆括号时不需要转义了，还支持了问号、加号 和 多选分支。

现在使用的 Linux 发行版，大多都集成了 GNU 套件。GNU 在实现 POSIX 标准时，做了一定的扩展，主要有以下三点扩展。

- **GNU BRE** 支持了 +、?，但转义了才表示特殊含义，即需要用+、\?表示。
- **GNU BRE** 支持管道符多选分支结构，同样需要转义，即用 |表示。
- **GNU ERE** 也支持使用反引用，和 BRE 一样，使用 \1、\2...\9 表示。

正则表达式特性	BRE标准	ERE标准
点号.、^、\$、[...]、[^...]	✓	✓
“任意数目”量词*	✓	✓
+和?量词	✗ (GNU BRE支持)	✓
区间量词	\{min,max\}	{min,max}
圆括号分组	\(...\)	(...)
量词可否限定圆括号分组	✓	✓
捕获文本引用	\1到\9	✗ (GNU ERE支持)
多选分支结构	✗ (GNU BRE支持)	✓

POSIX 字符组

POSIX 流派还有一个特殊的地方，就是有自己的字符组，叫 POSIX 字符组。这个类似于 `\d` 表示数字，`\s` 表示空白符等，POSIX 中也定义了一系列的字符组。具体的清单和解释如下所示：

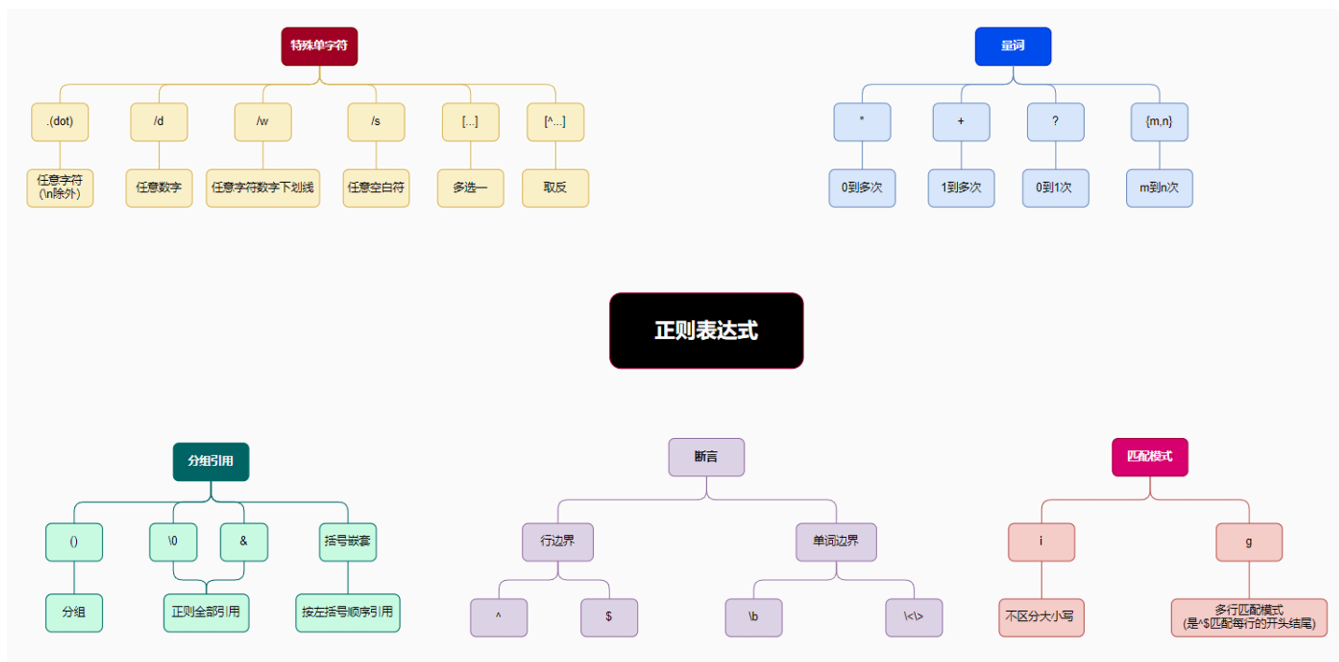
POSIX字符组	解释	等价表示	备注
<code>[[:alnum:]]</code>	数字和字母	<code>[0-9A-Za-z]</code>	比 <code>\w</code> 少了下划线
<code>[[:alpha:]]</code>	字母	<code>[A-Za-z]</code>	
<code>[[:ascii:]]</code>	ASCII	<code>[\x00-\x7F]</code>	
<code>[[:blank:]]</code>	空格和制表符	<code>[\t]</code>	
<code>[[:cntrl:]]</code>	控制字符	<code>[\x00-\x1F\x7F]</code>	
<code>[[:digit:]]</code>	数字	<code>[0-9]</code>	<code>\d</code>
<code>[[:graph:]]</code>	可见字符	<code>[!-~]</code> <code>[A-Za-z0-9!"#\$%&'()*+,\-./:;<=>?</code> <code>@[\]\]^_`{ }~]</code>	
<code>[[:lower:]]</code>	小写字母	<code>[a-z]</code>	
<code>[[:upper:]]</code>	大写字母	<code>[A-Z]</code>	
<code>[[:print:]]</code>	可打印字符	<code>[-~]</code> <code>[[:graph:]]</code>	比 <code>graph</code> 多了空格
<code>[[:punct:]]</code>	标点符号	<code>[-!/:-@[-`{-~]</code>	
<code>[[:space:]]</code>	空白符号	<code>[\t\n\v\f\r]</code>	
<code>[[:xdigit:]]</code>	16进制数字	<code>[0-9A-Fa-f]</code>	

PCRE 流派

除了 POSIX 标准外，还有一个 Perl 分支，也就是我们现在熟知的 PCRE。随着 Perl 语言的发展，Perl 语言中的正则表达式功能越来越强悍，为了把 Perl 语言中正则的功能移植到其他语言中，PCRE 就诞生了。

目前大部分常用编程语言都是源于 PCRE 标准，这个流派显著特征是有 `\d`、`\w`、`\s` 这类字符组简记方式。

正则表达式元字符



单字符匹配

```

.          # 任意单个字符（\n除外）
[...]     # 指定范围的字符，即多选一
[^...]    # 不在指定范围的字符，一样是多选一

# ERE | BRE
[:alnum:] # 数字和字母
[:alpha:] # 代表任何英文大小写字符，亦即 A-Z, a-z
[:lower:] # 小写字母, 示例: [:lower:], 相当于 [a-z]
[:upper:] # 大写字母
[:blank:] # 空白字符（空格和制表符）
[:space:] # 水平和垂直的空白字符（比[:blank:]包含的范围广）
[:digit:] # 十进制数字
[:xdigit:] # 十六进制数字
[:punct:] # 标点符号

# PCRE
\d        # 数字
\D        # 非数字
\w        # 数字，字母，下划线
\W        # 非数字，字母，下划线
\s        # 空白字符
\S        # 非空白符

```

POSIX ERE 标准本身 **不支持** `\w`、`\s`、`\d`，但：

GNU grep 的实现在某些环境下允许 `\w`、`\s`、`\d` 等 Perl 风格写法，即使使用的是 `-E`（ERE 模式），也可能生效。

实测 `grep -E` 不支持 `\d`，支持 `\w` 和 `\s`

因此，为跨平台场景考虑，不建议扩展正则使用 `\d, \w, \s`，建议使用 `[:alnum:]` 等替代

量词即匹配次数

<code>*</code>	# 匹配前面字符任意次
<code>?</code>	# 0或1次
<code>+</code>	# 1次或多次
<code>{n}</code>	# 匹配n次
<code>{m,n}</code>	# 至少m，至多n次

位置锚定

<code>^</code>	# 行首
<code>\$</code>	# 行尾
<code>\<, \b</code>	# 词首
<code>\>, \b</code>	# 词尾

分组其他

<code>()</code>	# 后向引用: <code>\1, \2, ...</code> 注意: <code>\0</code> 表示正则表达式匹配的所有字符，重点观察左括号
<code>(?:)</code>	# 非捕获分组，只是把里面的正则当作一个整体来控制重复次数或逻辑，不存储匹配内容到捕获组
	# 这样可以减少内存占用，并且避免多余的捕获编号
<code> </code>	# 或者
<code>a b</code>	# a或b
<code>C cat</code>	# C或cat
<code>(C c)at</code>	# Cat或cat

匹配模式

# 仅适用于PCRE	
<code>g (?g)</code>	# 全局匹配
<code>i (?i)</code>	# 忽视大小写

示例1

```
root@localhost:~# grep -P "(?i)super" /etc/passwd
root:x:0:0:Super User:/root:/bin/bash
```

示例2: 匹配IP

```
[root@ubuntu2204 ~]# ifconfig | grep -Po "((25[0-5])|(2[0-4][0-9])|(1[0-9][0-9])|([1-9]?[0-9]))\.{3}((25[0-5])|(2[0-4][0-9])|(1[0-9][0-9])|([1-9]?[0-9]))" | head -n1
10.0.0.202
```

示例3：排除空行和#开头的行

```
root@localhost:~# grep -v '^#' /etc/fstab | grep -v ^$
UUID=664b836b-0ac3-4d1c-b01b-cc543f2ec882 / xfs defaults 0
0
UUID=104ac312-22af-4143-8d75-6a9396e0f536 /boot xfs defaults 0
0
UUID=de822951-49ea-4cc2-ba02-0edea42f4ea5 none swap defaults 0
0

root@localhost:~# grep -Ev "^(#|$)" /etc/fstab
UUID=664b836b-0ac3-4d1c-b01b-cc543f2ec882 / xfs defaults 0
0
UUID=104ac312-22af-4143-8d75-6a9396e0f536 /boot xfs defaults 0
0
UUID=de822951-49ea-4cc2-ba02-0edea42f4ea5 none swap defaults 0
0
```

示例4：统计英文文章的单词数量，并从大到小排列

```
root@localhost:~# grep -oP "[A-Za-z]+(['-][A-Za-z]+)*" word.txt
```

文本处理三剑客

文本处理三剑客之 grep

grep: Global search REgular expression and Print out the line

作用：文本搜索工具，根据用户指定的“模式”对目标文本逐行进行匹配检查；打印匹配到的行

模式：由正则表达式字符及文本字符所编写的过滤条件

帮助

格式及常用选项

```
grep [OPTIONS...] PATTERN [FILE...]
```

常用选项

<code>-E --extended-regexp</code>	# 使用ERE, 相当于egrep
<code>-P --perl-regexp</code>	# 支持Perl格式的正则表达式
<code>-e --regexp=PATTERN</code>	# 实现多个选项间的逻辑or关系, 如: <code>grep -e 'cat' -e 'dog' file</code>
<code>-i --ignore-case</code>	# 忽略字符大小写
<code>-v --invert-match</code>	# 显示没有被匹配上的行, 即取反
<code>-n --line-number</code>	# 显示匹配的行号
<code>-q --quiet --silent</code>	# 静默模式, 不输出任何信息, 结果要从变量 <code>\$?</code> 拿
<code>-r --recursive</code>	# 递归目录, 但不处理软链接
<code>-m --max-count=N</code>	# 只匹配N行, 是行, 不是次数, 一行可能匹配两个, 但是, 这里是行
<code>-c --count</code>	# 统计匹配的行数, 是行数, 一行可以匹配一次到多次
<code>-A --after-context=N</code>	# 显示匹配到的字符串所在的行及其后n行
<code>-B --before-context=N</code>	# 显示匹配到的字符串所在的行及其前N行
<code>-C --context=N</code>	# 显示匹配到的字符串所在的行及其前后各N行

示例：标准输入

```
root@localhost:~# grep hello
123hello123
123hello123
```

示例：处理文件

```
root@localhost:~# grep root /etc/passwd
root:x:0:0:Super User:/root:/bin/bash
operator:x:11:0:operator:/root:/usr/sbin/nologin
```

示例：管道

```
root@localhost:~# cat /etc/passwd|grep root
root:x:0:0:Super User:/root:/bin/bash
operator:x:11:0:operator:/root:/usr/sbin/nologin
```

示例：取前三行


```
root@localhost:~# grep -m 3 bin /etc/passwd
root:x:0:0:Super User:/root:/bin/bash
bin:x:1:1:bin:/bin:/usr/sbin/nologin
daemon:x:2:2:daemon:/sbin:/usr/sbin/nologin
```

示例：取反，取不匹配的行

```
root@localhost:~# grep -v nologin /etc/passwd
root:x:0:0:Super User:/root:/bin/bash
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
magedu:x:1000:1000:magedu:/home/magedu:/bin/bash
```

不看注释行

```
root@localhost:~# grep -v "#" /etc/fstab
```

UUID=664b836b-0ac3-4d1c-b01b-cc543f2ec882 /	xf	default	0
0			
UUID=104ac312-22af-4143-8d75-6a9396e0f536 /boot	xf	default	0
0			
UUID=de822951-49ea-4cc2-ba02-0edea42f4ea5 none	swap	default	0
0			

范例：不区分大小写

```
root@localhost:~# grep -i ROOT /etc/passwd
root:x:0:0:Super User:/root:/bin/bash
operator:x:11:0:operator:/root:/usr/sbin/nologin
```

示例：显示行号

```
root@localhost:~# grep -n bash /etc/passwd
1:root:x:0:0:Super User:/root:/bin/bash
37:magedu:x:1000:1000:magedu:/home/magedu:/bin/bash
```

示例：显示匹配的行数

```
root@localhost:~# grep -c bash /etc/passwd
2
```

示例：仅显示匹配的内容

```
root@localhost:~# grep -o root /etc/passwd
root
root
root
```

示例：静默模式

```
root@localhost:~# grep -q root /etc/passwd
root@localhost:~# echo $?
0

root@localhost:~# grep -q roo00t /etc/passwd
root@localhost:~# echo $?
1
```

示例：显示匹配到的行及后两行

```
root@localhost:~# grep -A 2 root /etc/passwd
root:x:0:0:Super User:/root:/bin/bash
bin:x:1:1:bin:/bin:/usr/sbin/nologin
daemon:x:2:2:daemon:/sbin:/usr/sbin/nologin
--
operator:x:11:0:operator:/root:/usr/sbin/nologin
games:x:12:100:games:/usr/games:/usr/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/usr/sbin/nologin
```

范例：显示匹配到的行及前两行

```
root@localhost:~# grep -B 2 root /etc/passwd
root:x:0:0:Super User:/root:/bin/bash
--
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/usr/sbin/nologin
operator:x:11:0:operator:/root:/usr/sbin/nologin
```

范例：显示匹配到的行及前后各两行

```
root@localhost:~# grep -C 2 root /etc/passwd
root:x:0:0:Super User:/root:/bin/bash
bin:x:1:1:bin:/bin:/usr/sbin/nologin
daemon:x:2:2:daemon:/sbin:/usr/sbin/nologin
--
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/usr/sbin/nologin
operator:x:11:0:operator:/root:/usr/sbin/nologin
games:x:12:100:games:/usr/games:/usr/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/usr/sbin/nologin
```

示例：显示匹配 root 的行或匹配 bash 的行

```
root@localhost:~# grep -e root -e bash /etc/passwd
root:x:0:0:Super User:/root:/bin/bash
operator:x:11:0:operator:/root:/usr/sbin/nologin
magedu:x:1000:1000:magedu:/home/magedu:/bin/bash
```

示例：递归匹配

```
root@localhost:~# grep -r root /etc/*
/etc/aliases:postmaster:      root
/etc/aliases:bin:             root
.....
```

示例：命令行展开

```
root@localhost:~# grep $(whoami) /etc/passwd
root:x:0:0:Super User:/root:/bin/bash
operator:x:11:0:operator:/root:/usr/sbin/nologin
```

示例：取CPU核数

```
root@localhost:~# grep -c processor /proc/cpuinfo
2
```

范例：面试题，算出所有人的年龄总和

```
root@localhost:~# grep -Eo "[0-9]+" age.txt | tr '\n' + | grep -Eo "[0-9]" | bc
60
```

